

# 江南大学大学生创新创业训练计划

## 《 智能发酵食品酿造过程数字孪生 》

### 成果佐证材料

一、GitHub 项目源码仓库：<https://github.com/fanhua317/Digital-Twin-for-Smart-Fermented-Food-Fermentation>

二、系统文档佐证：项目总 README、frontend/src/components/3d/README.md 三维数字孪生模块说明、任务记录与调试验证记录。

三、前端功能佐证：Dashboard、PitMonitor、DeviceMonitor、AlarmCenter、ProductionManage、DigitalTwin 六个页面截图；ECharts 图表、窖池热力图、设备趋势图、告警处理流程和生产批次页面截图。

四、3D 仿真佐证：Scene.tsx、AGVModel、DeviceModel、PitModel、DistillationTower、SimulationPanel、GlobalSimulationRunner 等源码截图；5 台 AGV 路径动画、设备物料状态、输送管道、蒸馏塔和物料交易日志运行截图。

五、后端与接口佐证：Spring Boot 后端项目结构、实体模型、REST API、WebSocket、SimulatorService、热力图缓存、告警/生产接口对齐和 Docker Compose 部署配置截图。

江南大学

二〇二六年五月

# 一、GitHub 项目源码仓库

仓库地址：<https://github.com/fanhua317/Digital-Twin-for-Smart-Fermented-Food-Fermentation>

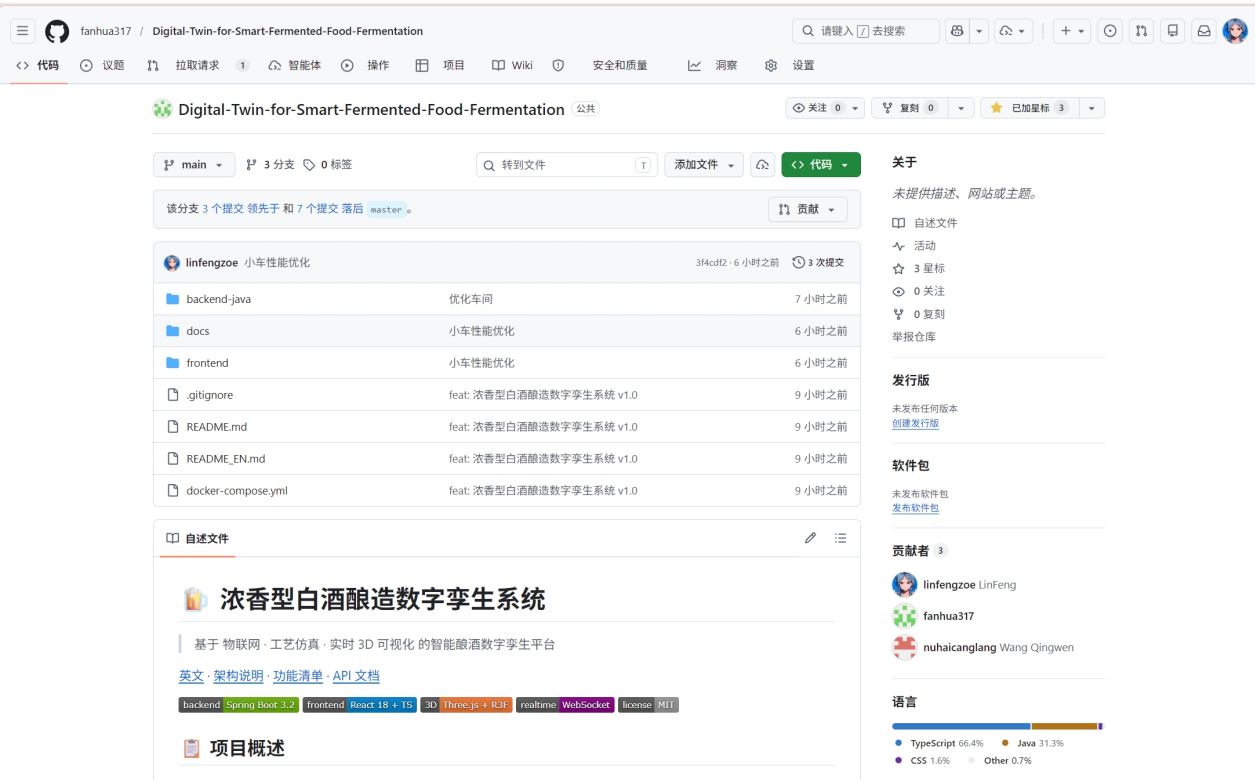


图 1-1 GitHub 项目源码仓库

## 二、系统文档佐证

### 2.1 三维数字孪生模块说明（3d/README.md）

基于 React Three Fiber + Three.js 0.182 实现的酿酒车间 3D 数字孪生场景，所有元素由后端 ProcessSimulationService 通过 WebSocket sim\_snapshot 帧实时驱动。主要文件：Scene.tsx（主场景）、PitModel.tsx（100 窖池实例化）、AGVModel.tsx（8 台 AGV 时间基插值动画）、DeviceModel.tsx（5 核心设备）、DistillationTower.tsx（蒸馏塔+摘酒动画）、SimulationController.tsx（7 步工艺导览）、PerformanceAnalysis.tsx（OEE/出酒率实时图表）。数据流：ProcessSimulationService → WebSocket /ws/realtime → useWebSocket hook → Zustand store → 3D 组件。

### 三、前端功能佐证

#### 3.1 综合监控中心（Dashboard）

4 个实时 KPI（窖池总数 100/设备总数 53/活跃告警/累计产量）+ 窖池状态分布饼图 + 设备状态分布饼图 + 24 小时告警趋势折线图 + 生产批次进度条，通过 WebSocket 实时同步。



图 3-1 综合监控中心 Dashboard

#### 3.2 窖池监控（Pit Monitor）

100 个窖池温度热力图 + 窖池列表（工艺阶段/糟醅类型/状态/发酵天数），支持过滤搜索分页。

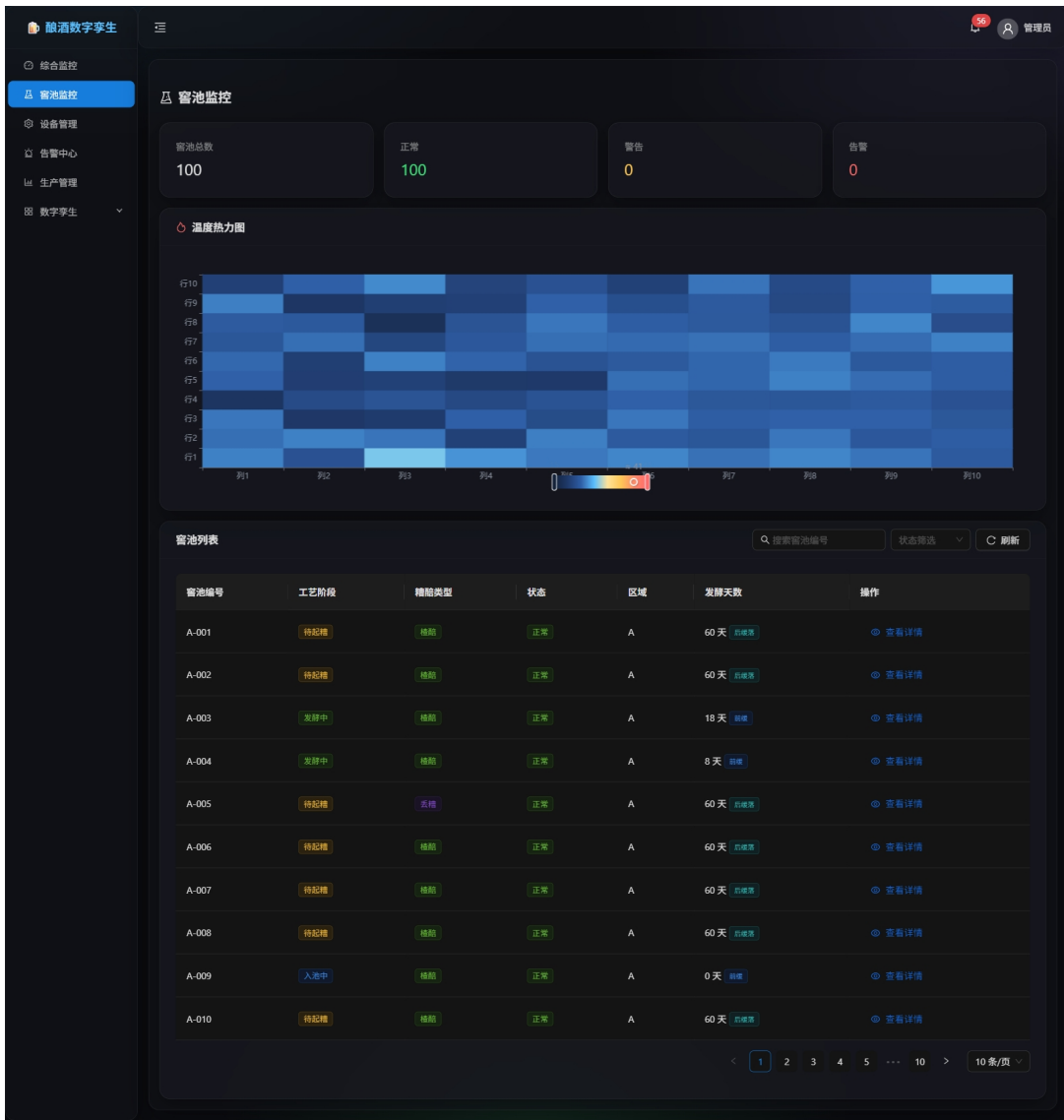


图 3-2 窖池监控（热力图 + 100 窖池列表）

3.3 窖池详情

点击查看详情：13 项工艺参数（入窖温度/水分/酸度/淀粉/粮糠比/粮曲比/粮醅比等）+ 实时传感器（温度/湿度/pH/酒精含量）+ 24 小时历史趋势折线图。



图 3-3 窖池详情弹窗（A-001，13 工艺参数 + 传感器历史曲线）

### 3.4 设备管理（Device Monitor）

53 台设备（5 核心+8AGV+40 辅助）+ 设备类型分布饼图 + 维护提醒进度条 + 状态色卡（运行中/停机/故障），支持查看单设备运行历史和告警历史。

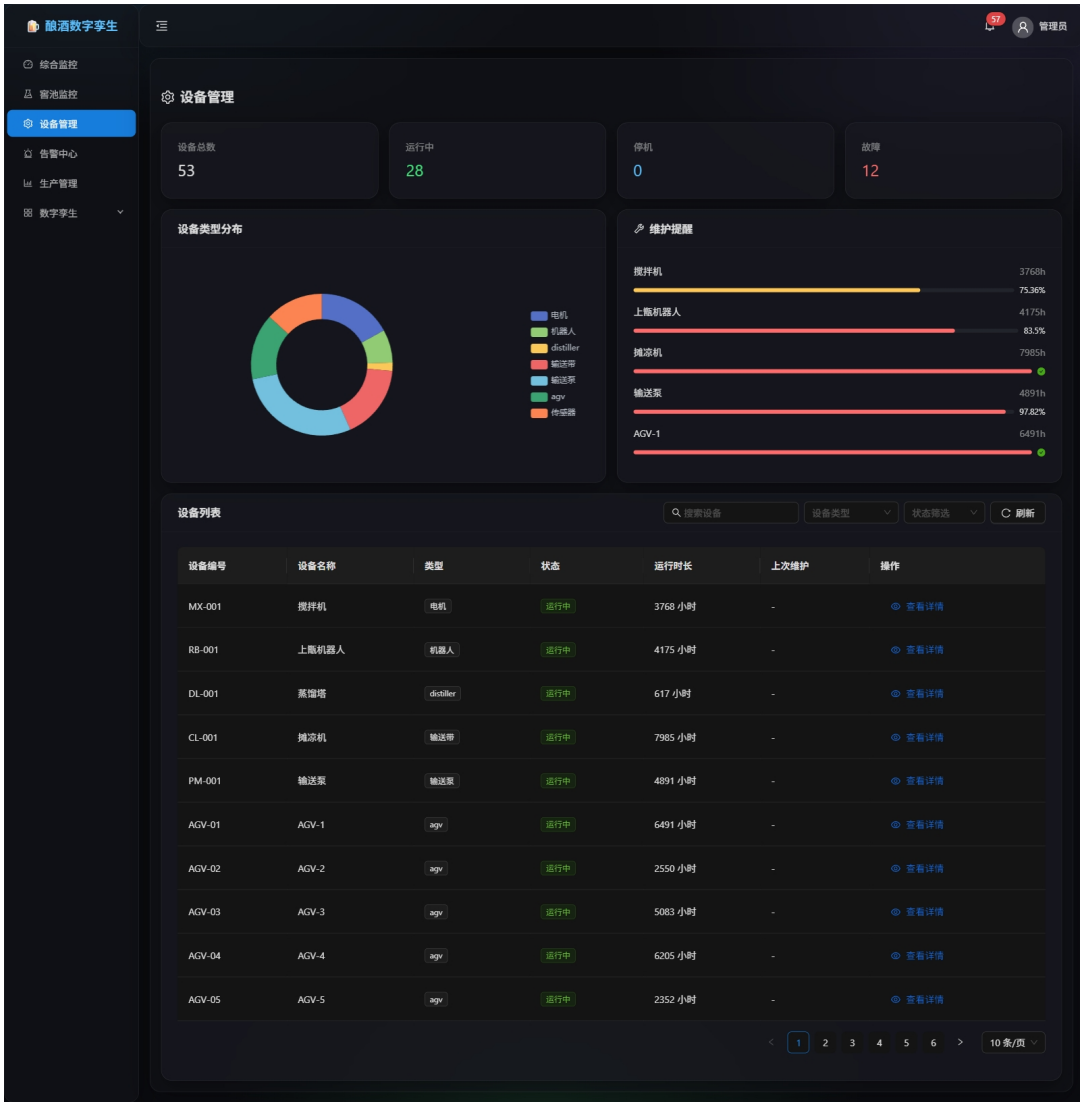


图 3-4 设备管理（53 台设备 + 类型分布 + 维护提醒）

3.5 告警中心（Alarm Center）

活跃告警 57 条，CRITICAL/WARNING/INFO 多级别，告警级别分布饼图 + 告警列表（来源/类型/信息/时间），支持一键解决和批量操作。



图 3-5 告警中心（57 条活跃告警 + 级别分布）

3.6 生产管理（Production Manage）

生产批次列表（批次编号/目标产量/状态/创建时间）+ 7 日产量趋势图 + 三联储罐存量（头酒 5%/中段 85%/尾酒 10%）+ 工艺参数面板（制曲/发酵/蒸馏关键参数）。

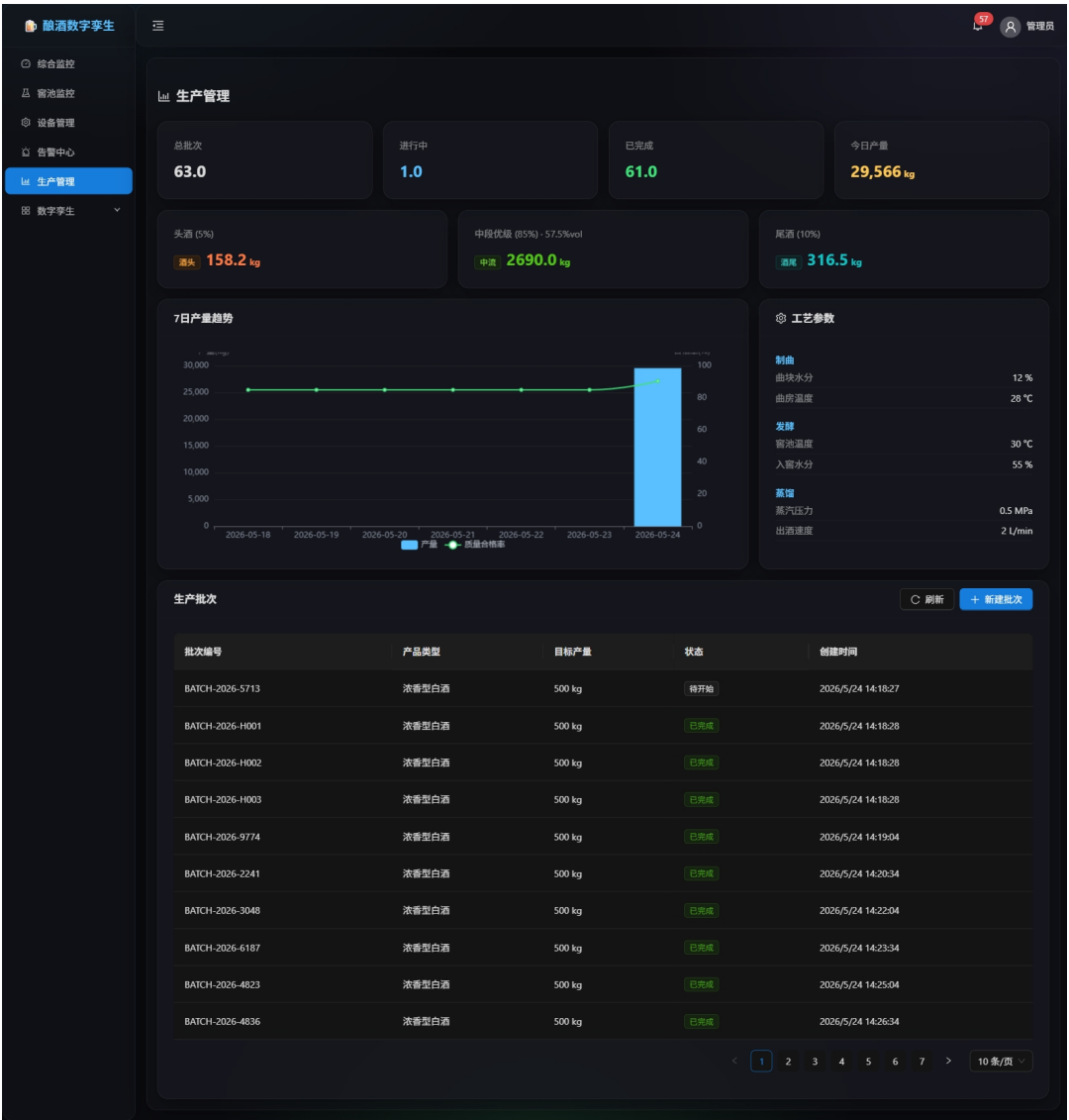


图 3-6 生产管理（批次列表 + 产量趋势 + 储罐存量）

### 3.7 数字孪生仿真模拟页

7 步工艺导览（起糟→配料→上甑→蒸馏→出甑→摊凉加曲→入池发酵）+ 暂停/继续/重置控制条 + 步骤自动定位摄像机 + 3D 场景实时联动。

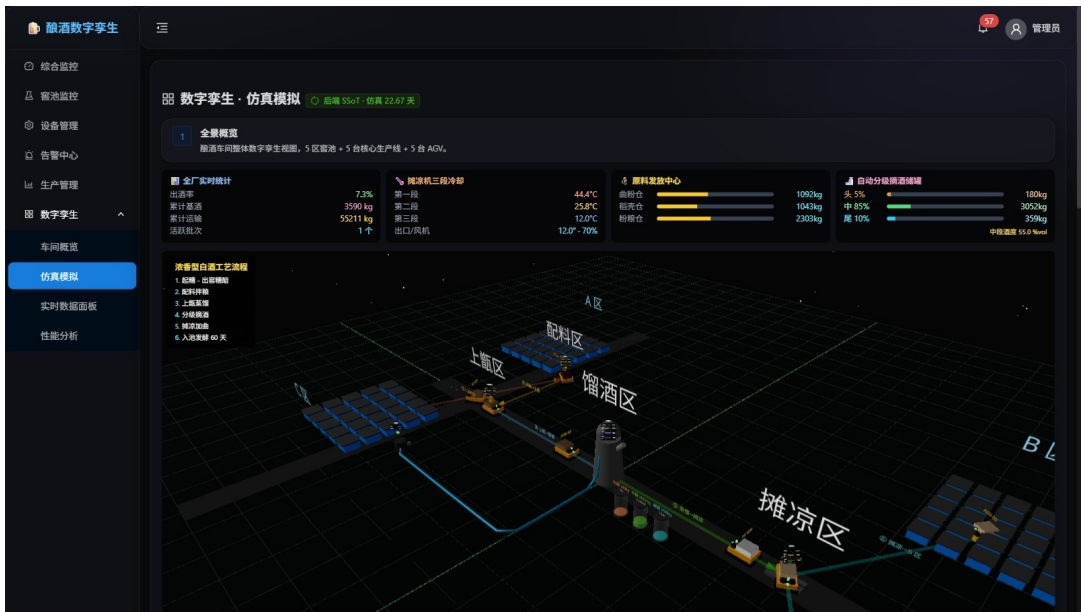


图 3-7 仿真模拟页（7 步工艺导览 + 3D 场景联动）

### 3.8 实时数据面板

AGV 实时状态（位置/任务/货物/重量/温度/pH）+ 设备物料流量 + 原料仓库存 + 储罐蓄酒量，每 500ms WebSocket 推送同步。

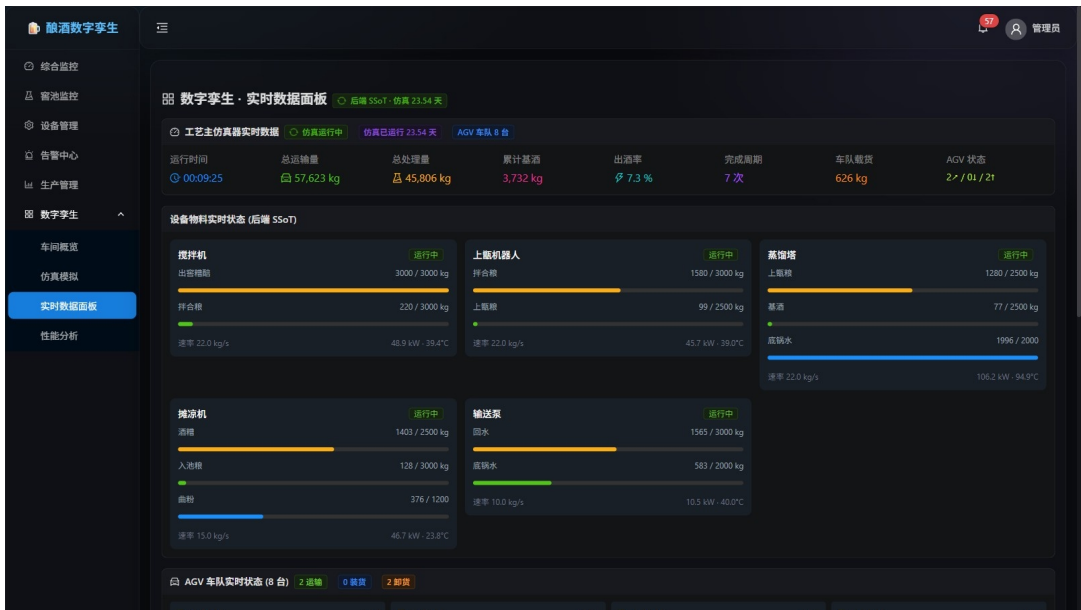


图 3-8 实时数据面板（AGV 状态 + 设备物料 + 原料仓）

### 3.9 性能分析

OEE 综合设备效率、可用率、能耗效率、出酒率雷达图及趋势图实时计算展示。



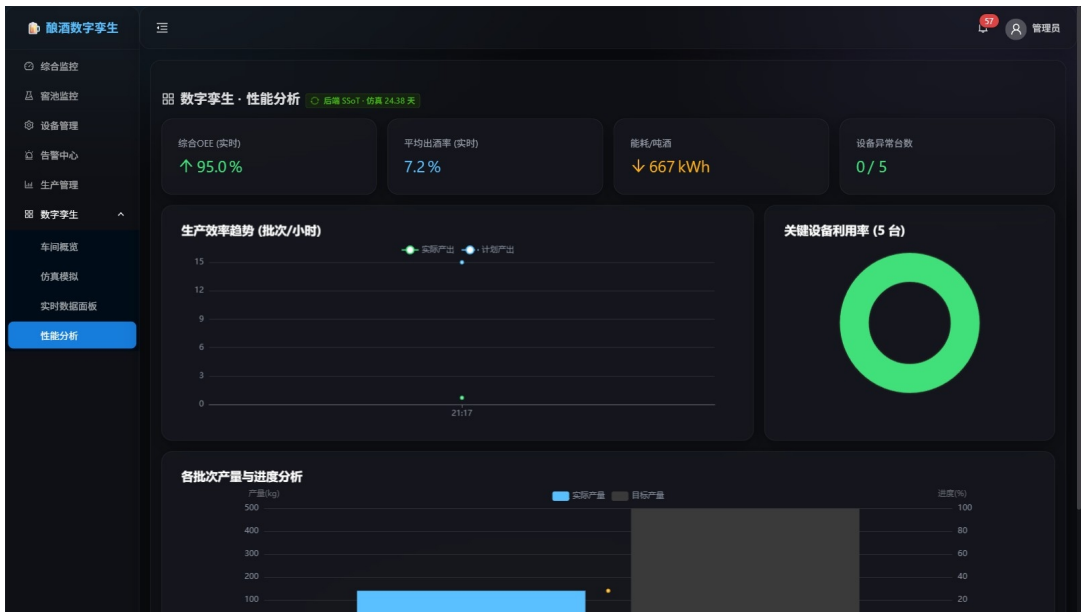


图 3-9 性能分析（OEE / 能耗 / 出酒率）

## 四、3D 仿真佐证

### 4.1 3D 车间全景运行截图

8 台 AGV 同时运行时间基插值动画，500ms/tick，60fps 无卡顿。工艺链路：A/C 区窖池 → 搅拌机 → 上甑机器人 → 蒸馏塔 → 摊凉机 → B/D 区窖池。画布外 HUD 实时显示全厂总览/摊凉机三段温度/原料三仓/三联储罐数据。

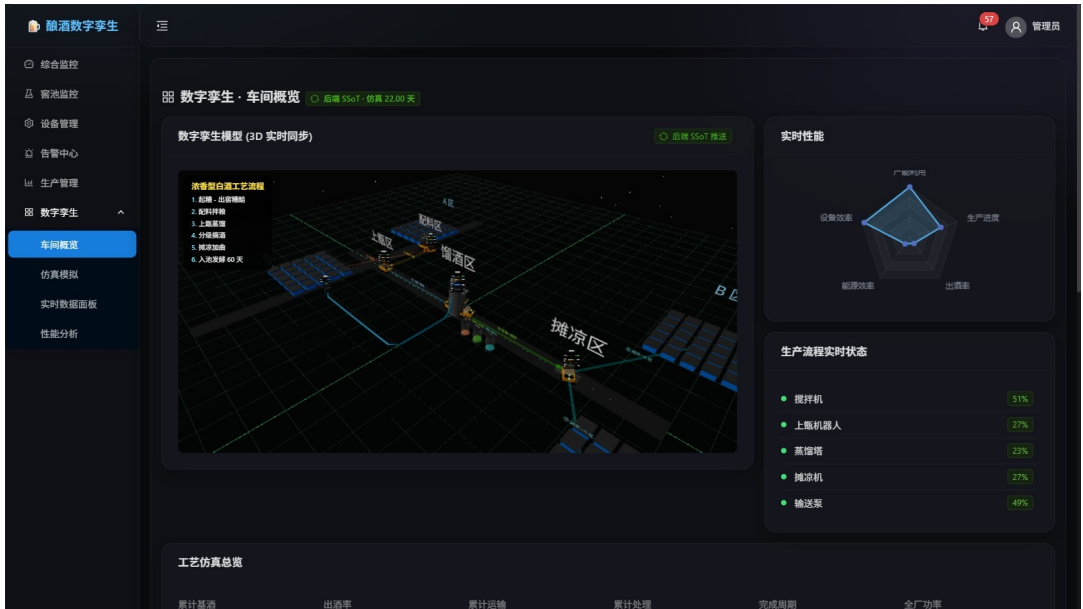


图 4-1 3D 车间全景（8 台 AGV 路径动画 + 100 窖池 + 5 核心设备）

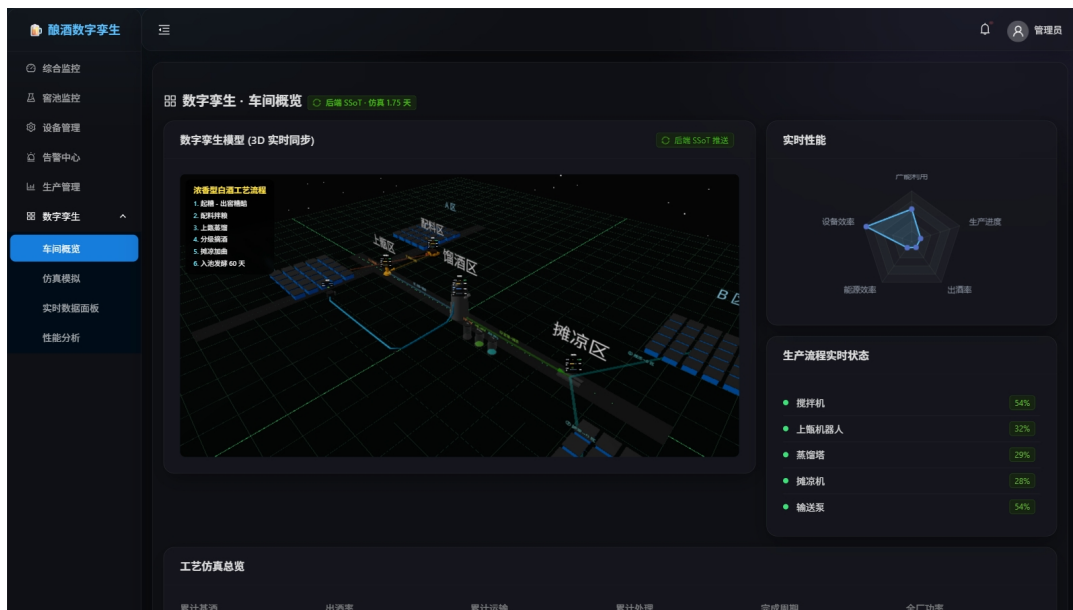


图 4-2 数字孪生车间概览（3D 实时同步 + 画布外 HUD + 工艺流程总览）

## 4.2 Scene.tsx — 3D 主场景

主场景容器，包含灯光、相机、100 窖池、5 核心设备、8AGV、输送管道、蒸馏塔及画布外 HUD。AGVModel 的 props 只有 agvId 和 isPlaying，挂载后永不变化，彻底消除 React re-render。

### Scene.tsx — AGV 常量化渲染 + WaterPipe 管道

```
// Scene.tsx - AGV 常量化渲染 (8台AGV永不触发re-render)
const AGV_CODES = ['AGV-01', 'AGV-02', 'AGV-03', 'AGV-04',
  'AGV-05', 'AGV-06', 'AGV-07', 'AGV-08']

// 改用常量数组渲染, props 永不变化 → AGVModel 零 re-render
{AGV_CODES.map(code => (
  <AGVModel key={code} agvId={code} isPlaying={isPlaying} />
))}

// WaterPipe: 输送泵→蒸馏塔管道, CatmullRomCurve3 流动动画
function WaterPipe() {
  const path = useMemo(() => new THREE.CatmullRomCurve3([...]), [])
  useFrame((_s, delta) => { textureRef.current.offset.x -= delta * 0.5 })
  return <mesh><tubeGeometry args={[path,16,0.2,6,false]} />...</mesh>
}
```

## 4.3 AGVModel.tsx — 时间基插值动画（架构核心）

AGV 动画完全在 useFrame 内完成，直读模块级 liveAgvCache 缓存，零 React reconciliation 参与位置更新，实现 60fps 平滑动画。

### AGVModel.tsx — useFrame 时间基插值核心逻辑

```
// AGVModel.tsx - useFrame 直读缓存, 绕过 React reconciliation
const BACKEND_TICK_MS = 600 // 匹配后端 500ms tick
const POSITION_TOLERANCE = 0.3

useFrame((_state, delta) => {
  const live = liveAgvCache[agvId] ?? EMPTY_LIVE // 直读缓存, 不触发渲染
  if (!isPlaying || live.stage === 'stopped') return

  const [nx, ny, nz] = live.position
  if (Math.abs(toPos.current.x - nx) > POSITION_TOLERANCE ||
    Math.abs(toPos.current.z - nz) > POSITION_TOLERANCE) {
    fromPos.current.copy(group.current.position)
    toPos.current.set(nx, ny, nz)
    tStartMs.current = performance.now()
  }
  // 时间基线性插值: 消除后端1s间隔导致的视觉跳帧
  const t = Math.min((performance.now() - tStartMs.current) / BACKEND_TICK_MS, 1)
  group.current.position.lerpVectors(fromPos.current, toPos.current, t)
})
export default memo(AGVModelImpl) // props不变 → 零re-render
```

#### 4.4 DeviceModel.tsx / DistillationTower.tsx / PitModel.tsx

DeviceModel: 5 核心设备（搅拌机/上甑机器人/蒸馏塔/摊凉机/输送泵），按物料输入/输出状态动态着色。

DistillationTower: 高塔 + 三联储罐（头酒/中段/尾酒）+ 摘酒液面动画 + PUMP 水管道。PitModel: 100 个窖池按 stage 和 grainType 颜色编码（空池/入池中/发酵中/待起糟/起糟中），React.memo 包裹，仅自身状态变化时重渲。

#### PitModel.tsx / DistillationTower.tsx — 关键片段

```
// PitModel.tsx - stage颜色编码
const STAGE_COLORS: Record<string, string> = {
  empty: '#1a1a2e',   filling: '#0d47a1',   fermenting: '#1565c0',
  ready: '#e65100',   discharging: '#bf360c'
}
export default memo(PitModelImpl)

// DistillationTower.tsx - 三联储罐液面平滑动画
useFrame(() => {
  const lvl = liquorStorage?.midLiquor / MAX_LIQUOR
  liquidMesh.current.scale.y = THREE.MathUtils.lerp(
    liquidMesh.current.scale.y, lvl, 0.05)
})
```

#### 4.5 SimulationPanel.tsx / GlobalSimulationRunner.tsx

SimulationPanel: 实时数据面板，展示 AGV 任务状态卡、设备物料流量、原料三仓（粉粮/稻壳/曲粉）库存、三联储罐蓄酒量，每 500ms WebSocket 推送刷新。GlobalSimulationRunner: 全局单例组件，在应用顶层挂载 WebSocket 连接，确保跨页面路由切换不断开连接，sim\_snapshot 帧持续更新 liveAgvCache。

#### GlobalSimulationRunner.tsx + useWebSocket.ts 节选

```

// GlobalSimulationRunner.tsx - 全文
import { useWebSocket } from '../hooks/useWebSocket'
export default function GlobalSimulationRunner() {
  useWebSocket() // 挂载全局 WS 单例, 跨路由保持连接
  return null
}

// useWebSocket.ts - sim_snapshot 处理 (节选)
ws.onmessage = (e) => {
  const msg = JSON.parse(e.data)
  if (msg.type === 'sim_snapshot') {
    updateLiveAgvCache(msg.agvs) // 直写缓存, 零React渲染
    if (now - lastSetTime > 2000) // 节流2s降低reconciliation频率
      setSimulation(msg)
  }
}
}

```

## 五、后端与接口佐证

### 5.1 Spring Boot 项目结构

#### backend-java 项目结构

```

backend-java/
├── pom.xml                Spring Boot 3.2.1 / JDK17 / Maven
├── Dockerfile
├── src/main/java/com/brewery/digitaltwin/
│   ├── BrewingDigitalTwinApplication.java
│   ├── entity/           JPA 实体
│   │   ├── Pit.java      窖池 (100个, 13工艺参数字段)
│   │   ├── Device.java   设备 (53个: 5核心+8AGV+40辅助)
│   │   ├── Alarm.java    告警
│   │   └── ProductionBatch.java
│   ├── service/
│   │   ├── ProcessSimulationService.java 工艺主仿真器 (500ms/tick)
│   │   └── DashboardService.java
│   ├── controller/      REST API (6个Controller)
│   ├── dto/             SimulationSnapshot / AGVState
│   ├── config/          WebSocketConfig / DataInitializer
│   └── websocket/       RealtimeWebSocketHandler

```

### 5.2 JPA 实体模型 — Pit.java (关键字段)

窖池实体, 13个工艺参数字段, 存储 100个窖池全量数据, 工艺阶段 (empty/filling/fermenting/ready/discharging) 驱动前端 3D 颜色编码。

#### entity/Pit.java — 关键字段

```

@Entity @Table(name="pits")
public class Pit {
    private String pitNo;           // 窖池编号 A-001
    private String zone;           // 区域 A/B/C/D
    private String stage;          // empty/filling/fermenting/ready/discharging
    private String grainCategory;  // 糟醅类型 zhapei/hongzao/diuzaao
    private Integer fermentationDay; // 发酵天数
    private Double entryTemperature; // 入窖温度 (°C)
    private Double entryMoisture;   // 入窖水分 (%) 典型51-55
    private Double entryAcidity;    // 入窖酸度 (mmol/10g) 典型1.4-2.5
    private Double entryStarch;     // 入窖淀粉 (%) 典型18-24
    private Double grainHullRatio;  // 粮糠比 (%) 典型22-27
    private Double grainKojiRatio;  // 粮曲比 (%) 典型20-25
    private Double grainMashRatio;  // 粮醅比 (1:N) 典型4-4.5
    private String fermentationPhase; // early/middle/late
}

```

### 5.3 REST API — SimulationController.java

提供工艺快照、暂停、继续、重置四个端点：前端在 WebSocket 未连通时通过 GET /snapshot 拉取首帧，也可手动调用 POST /pause、/resume、/reset 控制仿真进程。

controller/SimulationController.java

```

@RestController
@RequestMapping("/api/v1/simulation")
public class SimulationController {
    @GetMapping("/snapshot")
    public ApiResponse<SimulationSnapshot> snapshot() {
        return ApiResponse.success(simulationService.snapshot());
    }
    @PostMapping("/pause")
    public ApiResponse<Map<String,Object>> pause() {
        simulationService.pause();
        return ApiResponse.success(Map.of("paused", true));
    }
    @PostMapping("/resume") // 同理 ...
    @PostMapping("/reset")  // 重置仿真状态, AGV归位, 窖池重置
}

```

### 5.4 WebSocket 配置 + ProcessSimulationService（节选）

config/WebSocketConfig.java（前 30 行）

```
package com.brewery.digitaltwin.config;

import com.brewery.digitaltwin.websocket.RealtimeWebSocketHandler;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.socket.config.annotation.EnableWebSocket;
import org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;

@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {

    private final RealtimeWebSocketHandler webSocketHandler;

    public WebSocketConfig(RealtimeWebSocketHandler webSocketHandler) {
        this.webSocketHandler = webSocketHandler;
    }

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(webSocketHandler, "/ws/realtime", "/ws/**")
            .setAllowedOriginPatterns("*");
    }
}
```

service/ProcessSimulationService.java（前 40 行，含@Scheduled 配置）



```

package com.brewery.digitaltwin.service;

import com.brewery.digitaltwin.dto.AGVState;
import com.brewery.digitaltwin.dto.EquipmentState;
import com.brewery.digitaltwin.dto.RealtimeMessage;
import com.brewery.digitaltwin.dto.SimulationSnapshot;
import com.brewery.digitaltwin.entity.Device;
import com.brewery.digitaltwin.entity.DeviceData;
import com.brewery.digitaltwin.entity.Pit;
import com.brewery.digitaltwin.entity.ProductionBatch;
import com.brewery.digitaltwin.repository.DeviceDataRepository;
import com.brewery.digitaltwin.repository.DeviceRepository;
import com.brewery.digitaltwin.repository.PitRepository;
import com.brewery.digitaltwin.repository.ProductionBatchRepository;
import com.brewery.digitaltwin.websocket.RealtimeWebSocketHandler;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

/**
 * 工艺主仿真器：白酒生产数字孪生的单一数据源（SSoT）。
 *
 * <p>每秒一拍，按工厂级速率推进设备物料、AGV 物流、窖池阶段、生产批次。
 * 严格遵守物料守恒：source.output -= amount → AGV.weight = amount → AGV.weight = 0,
 * target.input += amount。</p>
 *
 * <p>仿真时间换算：1 实秒 = 1 仿真分钟（TIME_SCALE=60），演示 60 秒可见 1 个发酵小时。</p>
 *
 * <p>工艺路线：起糟（窖池→搅拌机）→ 拌粮（搅拌机→上甑机器人）→ 上甑（机器人→蒸馏塔）
 * → 蒸馏出酒（塔→摊凉机）→ 摊凉加曲（摊凉机→入池）→ 发酵（窖池，60 仿真天）。</p>

```

## 5.5 Docker Compose 部署配置 + application.yml

一键 docker-compose up 启动前后端；H2 数据库文件持久化挂载（brewing-data 卷），容器重启数据不丢失；前端 Nginx 反代，后端 Spring Boot 端口 8000，仿真 tick 间隔 500ms 可配置。

**docker-compose.yml（全文）**

```

version: '3.8'

services:
  backend:
    build:
      context: ./backend-java
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      # 容器内允许的前端域名（逗号分隔）· 生产时改成你的真实域名，例：https://twin.example.com
      - APP_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
    volumes:
      # 持久化 H2 数据库，容器重启不丢数据
      - brewing-data:/app/data
    restart: always

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:80"
    depends_on:
      - backend
    restart: always

volumes:
  brewing-data:

```

## application.yml（关键配置节选）

```

server:
  port: 8000

spring:
  application:
    name: brewing-digital-twin

  datasource:
    url: jdbc:h2:file:./data/brewing;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE;MODE=LEGACY
    driver-class-name: org.h2.Driver
    username: sa
    password:

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false
    open-in-view: false
    properties:
      hibernate:
        format_sql: false
        dialect: org.hibernate.dialect.H2Dialect

  h2:
    console:
      enabled: true
      path: /h2-console

# ===== 应用业务配置 =====
app:

```



附：REST API 速查表

| 模块 | 方法   | 路径                              | 说明                        |
|----|------|---------------------------------|---------------------------|
| 仿真 | GET  | /api/v1/simulation/snapshot     | 工艺快照（AGV/设备/窖池/储罐全量）      |
| 仿真 | POST | /api/v1/simulation/pause        | 暂停仿真                      |
| 仿真 | POST | /api/v1/simulation/resume       | 继续仿真                      |
| 仿真 | POST | /api/v1/simulation/reset        | 重置仿真，AGV 归位               |
| 监控 | GET  | /api/v1/dashboard/stats         | 全厂统计 KPI                  |
| 监控 | GET  | /api/v1/dashboard/overview      | 生产进度/设备效率概览               |
| 窖池 | GET  | /api/v1/pits                    | 100 个窖池列表                 |
| 窖池 | GET  | /api/v1/pits/{id}/sensors       | 单池传感器历史数据                 |
| 窖池 | GET  | /api/v1/pits/heatmap            | 100 窖池温度热力图数据             |
| 设备 | GET  | /api/v1/devices                 | 53 台设备列表                  |
| 设备 | GET  | /api/v1/devices/{id}/data       | 设备运行历史                    |
| 告警 | GET  | /api/v1/alarms/active           | 当前活跃告警                    |
| 告警 | PUT  | /api/v1/alarms/{id}/resolve     | 处理告警                      |
| 生产 | GET  | /api/v1/production/batches      | 生产批次列表                    |
| 生产 | POST | /api/v1/production/batches      | 新建生产批次                    |
| WS | WS   | ws://localhost:8000/ws/realtime | sim_snapshot 每 500ms 广播一帧 |

WebSocket sim\_snapshot 数据格式

WebSocket sim\_snapshot 帧结构（每 500ms 推送）

```
{
  "type": "sim_snapshot",
  "agvs": [{ "id": "AGV-01", "x": 12.5, "z": -8.3, "status": "moving",
             "cargo": "grain", "weight": 850, "temperature": 25.4, "ph": 4.2 }],
  "equipments": { "MIXER": { "inputLevel": 1800, "outputLevel": 250, "running": true },
                  "DISTILLER": { "inputLevel": 500, "running": true } },
  "pits": [{ "pitNo": "A-001", "stage": "fermenting", "fermentDay": 23 }],
  "liquorStorage": { "headLiquor": 820, "midLiquor": 6430, "tailLiquor": 530 },
  "rawMaterials": { "grainPowder": 4200, "riceCrust": 1800, "starter": 1100 }
}
```